

Markov Chain Modeling of a Search-and-Rescue Robot

From Discrete-Time Navigation to Continuous-Time Mission Dynamics

Ana Mafalda Carmo, Filipe Wu, Francisco Silva, Rafaela Pinto
Grupo1 – PEA – MECAD, 2025/2026
Faculdade de Ciências, Universidade do Porto

Abstract—This report presents a Streamlit application for modeling a search-and-rescue robot operating in a hazardous grid environment using Markov chains. The same mission is represented first as a discrete-time Markov chain (DTMC), where the robot moves step by step, and then as a continuous-time Markov chain (CTMC), where state changes occur after exponentially distributed waiting times. The app supports random and greedy policies, uncertain actuation, absorbing victim and hazard states, transition-matrix visualization, n -step prediction, absorbing-chain analysis, and continuous-time evolution through a generator matrix. The results show how Markov-chain tools can quantify mission success probability, expected time until absorption, risk of failure, and the temporal distribution of the robot over the grid.

Index Terms—Markov chain, absorbing chain, continuous-time Markov chain, rescue robotics, transition matrix, Streamlit visualization.

I. INTRODUCTION

Search-and-rescue robots are commonly deployed in uncertain environments where motion commands may fail because of wheel slip, irregular terrain, sensor error, or limited localization accuracy. Even in a simplified grid world, uncertainty makes deterministic path planning insufficient, motivating a probability-model view of the mission [2]: the robot may intend to move toward a victim, but the actual movement can lead to another cell or even a hazardous region.

The developed application studies this problem through a FrozenLake-inspired rescue interpretation based on the Gymnasium environment interface and its standard family of reinforcement-learning environments [6]. The agent is interpreted as a rescue robot, safe tiles as traversable terrain, holes as hazards or unstable floor regions, and the goal as the victim or rescue target. The application provides both an educational and analytical interface: it visualizes the grid, builds the Markov transition matrix, classifies communication classes, computes long-run absorption probabilities, and compares discrete-time and continuous-time mission dynamics.

The main objective is to show how a stochastic model can answer operational questions such as: Where is the robot likely to be after n steps? What is the probability of reaching the victim before entering a hazard? How many steps are expected before the mission ends? How does the answer change when transitions occur in continuous time rather than fixed steps?

II. BACKGROUND

A. Discrete-Time Markov Chain

Let X_t denote the robot state at discrete step t . In the proposed model, a state is only the grid position of the robot:

$$s_t = X_t = (r, c), \quad (1)$$

where r and c are the row and column of a cell. For an $m \times m$ grid, the Markov chain has m^2 states and a transition matrix $P \in \mathbb{R}^{m^2 \times m^2}$. The discrete-time formulation follows the standard Markov-chain model for stochastic processes [2]. The Markov property is

$$\Pr(X_{t+1} = s_j \mid X_t = s_i, X_{t-1}, \dots) = \Pr(X_{t+1} = s_j \mid X_t = s_i). \quad (2)$$

Each row of P is a probability distribution:

$$P_{ij} = \Pr(X_{t+1} = s_j \mid X_t = s_i), \quad \sum_j P_{ij} = 1. \quad (3)$$

The robot first chooses an intended action $a \in \{\text{up, down, left, right}\}$ according to a policy $\pi(a \mid s)$. Then actuation uncertainty is applied. If p is the probability of executing the intended movement, then

$$\Pr(a_{\text{actual}} = a_{\text{intended}}) = p, \quad (4)$$

while the remaining probability is distributed over the other directions:

$$\Pr(a_{\text{actual}} = b) = \frac{1-p}{3}, \quad b \neq a_{\text{intended}}. \quad (5)$$

The final transition probability is therefore obtained by marginalizing over intended and actual actions:

$$P_{ij} = \sum_a \pi(a \mid s_i) \sum_b \Pr(b \mid a) \mathbf{1}\{T(s_i, b) = s_j\}, \quad (6)$$

where $T(s_i, b)$ is the next grid cell after applying actual action b . If the action would leave the grid, the robot remains in the same cell, creating a self-transition.

B. Absorbing Markov Chains

Victim and hazard cells are modeled as absorbing states, following the usual absorbing-chain decomposition [1], [2]. Thus, if s_v is the victim and s_h is a hazard,

$$P_{vv} = 1, \quad P_{hh} = 1. \quad (7)$$

After reordering states so that transient states appear first and absorbing states last, the transition matrix has block form

$$P = \begin{pmatrix} Q & R \\ 0 & I \end{pmatrix}, \quad (8)$$

where Q contains transient-to-transient transitions and R contains transient-to-absorbing transitions. The fundamental matrix is

$$N = (I - Q)^{-1}. \quad (9)$$

The entry N_{ij} is the expected number of visits to transient state s_j when starting from transient state s_i . The expected number of steps until absorption is

$$\tau = N\mathbf{1}, \quad (10)$$

where $\mathbf{1}$ is a vector of ones. The absorption probability matrix is

$$B = NR. \quad (11)$$

In particular, the probability of reaching the victim before any hazard can also be obtained by solving

$$(I - Q)h_T = r, \quad (12)$$

where r is the column of R corresponding to the victim state.

C. Continuous-Time Markov Chain

The continuous-time model replaces fixed movement steps by random waiting times, using the standard CTMC generator formalism [3], [4]. It is defined by a generator matrix G , whose off-diagonal entries are transition rates:

$$g_{ij} = \lambda P_{ij}, \quad i \neq j, \quad (13)$$

where $\lambda > 0$ is the transition-rate parameter. The diagonal entries enforce zero row sums:

$$g_{ii} = -\sum_{j \neq i} g_{ij}. \quad (14)$$

Absorbing states have zero rows in G . The finite-time CTMC transition matrix is computed using the matrix exponential:

$$P(t) = e^{Gt} = I + Gt + \frac{(Gt)^2}{2!} + \frac{(Gt)^3}{3!} + \dots \quad (15)$$

Given an initial distribution π_0 , the distribution at time t is

$$\pi(t) = \pi_0 P(t). \quad (16)$$

III. PROPOSED SOLUTION

The proposed solution is an interactive Streamlit application organized into three tabs: overview, DTMC analysis and CTMC analysis. The sidebar controls define the experiment configuration: grid size (4×4 or 8×8), number of holes, policy type, intended-movement probability p , edge-visualization threshold, continuous-time rate λ , and random seed.

The DTMC module builds the transition matrix from the selected grid, policy, and motion uncertainty. Two policies are considered. The random policy assigns equal probability to valid intended actions. The greedy policy biases the intended action toward the victim. After the intended action is chosen, stochastic actuation converts it into an actual action using the probability model above.

TABLE I: Conceptual mapping between the application and the mathematical model.

Application element	Mathematical object or operation
Grid cell	State s_i
Robot position	Current random variable value X_t
Policy selector	Distribution $\pi(a s)$ over intended actions
Slip probability	Conditional distribution $\Pr(b a)$
Transition matrix tab	Matrix P and powers P^n
Absorbing analysis	Blocks Q, R , matrix N , vector τ , matrix B
Continuous-time tab	Generator G and transition matrix e^{Gt}
Animation	Evolution of π_n or $\pi(t)$ over the grid

The application then provides several layers of analysis. First, it shows the grid and the transition graph, where nodes represent states and directed edges represent nonzero transition probabilities. Second, it classifies the chain into communication classes and identifies absorbing, recurrent, and transient states. Third, it supports one-step simulation and full n -step prediction:

$$\pi_n = \pi_0 P^n. \quad (17)$$

Finally, the absorbing-chain module computes mission-level indicators: success-before-hazard probability, expected steps until absorption, the fundamental matrix N , and the absorption probability matrix B .

The CTMC module reuses the same discrete transition structure as an embedded chain and converts probabilities into rates through G [5]. This preserves the spatial transition logic while allowing the mission to be analyzed over real time rather than over movement counts. The app visualizes G , $P(t) = e^{Gt}$, and the evolution of success, failure, and survival probabilities.

A. Implementation Structure

The implementation separates the interface from the mathematical operations. The main Streamlit script manages user controls, session state, layout, and visualization, while utility modules implement the stochastic model construction and plotting functions. The discrete module builds the grid, extracts state positions, checks absorbing states, builds P , simulates a sampled robot step, computes π_n , classifies the chain, and performs absorbing-chain analysis. The continuous module converts P into G and computes $P(t)$ through the matrix exponential. The plotting module generates the grid renderings, graph visualizations, matrix heatmaps, probability grids, and animations.

This separation is important because each mathematical object has a clear role. The matrix P is the central DTMC object; all discrete predictions, class analysis, and absorbing-chain calculations are derived from it. The generator G is the central CTMC object; all continuous-time probability distributions are derived from e^{Gt} . The interface therefore acts mainly as an experimental layer around these two matrices.

IV. EXPERIMENTAL METHODOLOGY

Experiments are performed by selecting a grid configuration and comparing the resulting stochastic behavior under different control parameters. The basic workflow is: generate a grid with

TABLE II: Main experimental parameters exposed by the application.

Parameter	Meaning	Typical values
Grid size	Number of cells	$4 \times 4, 8 \times 8$
Holes	Hazard count	0 to $m^2 - 2$
Policy	Intended-action rule	Random, Greedy
p	Intended movement probability	0.25 to 1.0
λ	CTMC transition rate	0.1 to 5.0
n	DTMC prediction horizon	1 to 200 steps
t	CTMC prediction horizon	0 to 10 time units

a fixed seed; select a policy; choose p and λ ; build P ; analyze P^n and absorbing-chain quantities; construct G ; and analyze $P(t)$.

The main graphics included on this report are exported as screenshots from the Streamlit application. Fig. 1 summarizes the most important ones: the rescue grid, transition graph, transition-matrix heatmap, absorbing-chain block matrix, fundamental matrix, absorption probabilities, generator matrix, and continuous-time probability evolution.

The numerical procedure used by the app can be summarized as follows. First, all grid positions are enumerated and mapped to state indices. Second, for each non-absorbing state, the selected policy defines the intended-action distribution. Third, movement uncertainty is applied and probability mass is accumulated into the correct next-state columns of P . Fourth, absorbing rows are overwritten with identity rows. Finally, all analysis quantities are computed from P or from the generator G derived from P .

$$\text{state index}(r, c) = r \cdot m + c, \quad (18)$$

for an $m \times m$ grid. This indexing is used consistently in the transition matrices, heatmaps, communication-class table, and probability vectors. Because the same state order is preserved across the DTMC and CTMC modules, results from P , P^n , G , and $P(t)$ remain directly comparable.

V. EVALUATION & RESULTS

The app evaluates the model at three complementary levels. At the local level, each row of P explains the one-step behavior from a selected state. This is useful for debugging boundary effects, self-transitions, and the influence of the intended-movement probability. When $p = 1$, the actual movement always equals the intended movement. When $p < 1$, probability mass spreads to unintended directions, increasing the risk of entering hazards.

At the finite-horizon level, the application computes $\pi_n = \pi_0 P^n$. The corresponding heatmap identifies the row of P^n associated with the start state and the most likely final state after n steps. This gives a compact visual explanation of how a matrix power encodes all possible paths of length n .

At the mission-outcome level, absorbing-chain analysis provides the most relevant performance indicators. The success probability from the start state is obtained from the victim column of B or equivalently from the solution of $(I - Q)h_T = r$. The failure probability is $1 - h_{start}$ when the absorbing states are only victim and hazards. The vector $\tau = N\mathbf{1}$ gives the expected number of movement steps before either success or

TABLE III: Main outputs produced by the application.

Output	Interpretation
Transition graph	Visual map of possible state changes and probabilities.
P heatmap	One-step transition probabilities between all states.
P^n heatmap	Probability of reaching each state after n steps.
Class table	Recurrent, transient, closed, and absorbing classes.
$N = (I - Q)^{-1}$	Expected visits to transient states before absorption.
$\tau = N\mathbf{1}$	Expected steps until the mission ends.
$B = NR$	Probability of final absorption in victim or hazards.
G heatmap	CTMC instantaneous transition rates.
$P(t) = e^{Gt}$	State distribution after continuous time t .

failure. States with high τ_i indicate regions where the robot is expected to remain active for longer before absorption.

In the CTMC evaluation, the generator matrix G converts the DTMC into a time-indexed stochastic process. Larger values of λ make transitions occur more rapidly, so absorption probabilities are reached faster in real time. However, when G is constructed from the same spatial matrix P , changing λ changes the speed of the process, not the final absorption probabilities. Thus, the CTMC analysis is especially useful for mission-duration interpretation, while the DTMC analysis is more natural for movement-count interpretation.

VI. DISCUSSION

The discrete-time and continuous-time models answer related but different questions. The DTMC is appropriate when the mission is naturally counted in robot decisions or movement attempts. It explains how uncertainty accumulates over a fixed number of steps and gives direct access to P^n . The CTMC is more suitable when the timing of movements matters, because it models the waiting time before transitions and allows the distribution to be evaluated at arbitrary continuous times.

The absorbing-chain results are particularly important because they compress infinitely many possible trajectories into interpretable mission metrics. Instead of simulating many paths, the model computes exact success and failure probabilities through linear algebra. The fundamental matrix also provides diagnostic information: if certain transient states have high expected visit counts, they may correspond to slow, risky, or repeatedly visited regions of the environment.

The graphical interface improves interpretability. The grid view connects the mathematics to the rescue scenario, the transition diagram exposes the graph structure, the heatmaps reveal matrix patterns, and the animations distinguish between one sampled trajectory and the full probability distribution. This distinction is crucial: a simulation shows one possible mission, while the Markov chain describes all possible missions probabilistically.

A limitation is that the state only includes robot position. Real rescue robots may require richer state variables, such as orientation, battery level, sensor confidence, victim detection status, or terrain class. Adding these variables would increase

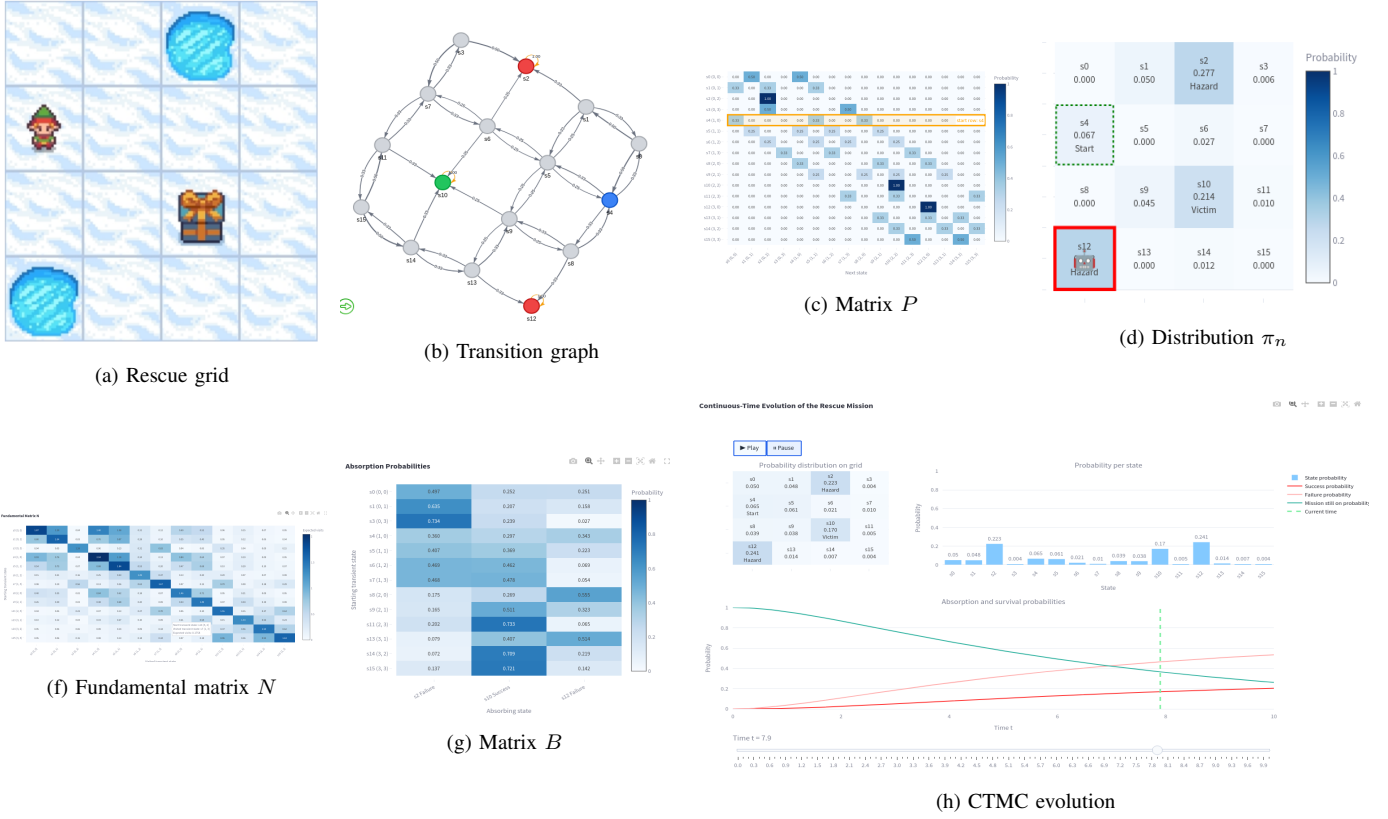


Fig. 1: Main visual outputs generated by the application: rescue environment, transition structure, finite-step distributions, absorbing-chain analysis, and continuous-time mission evolution.

the state space and therefore the size of P and G , but it would also make the model more realistic. For example, including battery level would allow the absorbing states to include both mission success, hazard failure, and energy depletion.

The greedy policy is also simple and does not optimize a formal cost function. It is useful for explaining the model because its behavior is intuitive, but it may be suboptimal in maps where the shortest direction to the victim passes close to hazards. A natural next step is to compare policies using success probability and expected absorption time, or to compute an optimal policy using dynamic programming. Nevertheless, the current model provides a clear basis for risk-aware mission analysis.

VII. CONCLUSION AND FUTURE WORK

This work developed an interactive Markov-chain application for analyzing a simplified search-and-rescue robot mission. The DTMC model describes step-by-step stochastic navigation, while the CTMC model extends the same transition structure to continuous-time dynamics. The application computes and visualizes transition matrices, state distributions, communication classes, absorbing-chain metrics, expected absorption times, and finite-time CTMC probabilities.

Future work should extend the state representation beyond position, incorporate learned or optimized policies, compare policies quantitatively over many generated maps, and add cost

functions for fuel, time, and risk. Another useful extension is to integrate reinforcement learning, where the Markov-chain analysis could be used to evaluate or explain policies learned by an agent.

REFERENCES

- [1] A. Yashinski, "Absorbing Markov chains," University of Maryland, MATH401 notes. [Online]. Available: https://www.math.umd.edu/~immortal/MATH401/book/ch_absorbing_markov_chains.pdf
- [2] A. Leon-Garcia, *Probability, Statistics, and Random Processes for Electrical Engineering*, 3rd ed. Upper Saddle River, NJ, USA: Pearson Education, 2008.
- [3] Stanford University, "Continuous time processes," STAT 217 notes, Chapter 3. [Online]. Available: <https://web.stanford.edu/class/stat217/Chapter3.pdf>
- [4] H. Pishro-Nik, "The generator matrix," *Introduction to Probability, Statistics, and Random Processes*. [Online]. Available: https://www.probabilitycourse.com/chapter11/11_3_3_the_generator_matrix.php
- [5] S. Lee, "Mastering embedded Markov chains," Number Analytics. [Online]. Available: <https://www.numberanalytics.com/blog/embedded-markov-chain-guide-actl-3102>
- [6] M. Towers *et al.*, "Gymnasium: A standard interface for reinforcement learning environments," *arXiv preprint arXiv:2407.17032*, 2024. [Online]. Available: <https://arxiv.org/abs/2407.17032>